

Apex is a tool-agnostic framework that brings project awareness, specialist routing, and built-in quality gates to any AI coding tool. It works with Cursor, GitHub Copilot, and Claude Code. The framework is the constant. the tool is interchangeable. One-time project setup. Centralized conventions. Unified behavior across every developer and every session.

APPROVALS & ACCESS

Before you can use the Apex Framework, you need approved access to at least one of these AI coding tools: **VS Code with GitHub Copilot**, **Cursor**, or **Claude Code**. You can use one, two, or all three. The framework works identically regardless of which tool you choose.

These tools require an enterprise license approved by Deloitte or the client. Reach out to your lead, manager, or DT team to request access.

Once you have access to at least one of these tools, you are ready to move on to setup.

SETUP

Setting up the Apex Framework involves two parts: installing your AI coding tool and then adding the framework to your project. Both are straightforward and only need to be done once.

Part 1: Install Your AI Coding Tool

Install the tool your team has been approved to use. If your team has access to more than one, you can install all of them. Each card below covers what the tool is, how to install it, how the framework works inside it, and where to find the official documentation.

VS Code with GitHub Copilot

A free, lightweight code editor from Microsoft. GitHub Copilot is an AI assistant that works as an extension inside VS Code.

- Slash commands are invoked by attaching command files via the # symbol in the chat
- Agents appear in an Agents dropdown in the Copilot chat panel. Select an agent (like Planner or Backend Dev) and it becomes your active assistant
- After an agent completes its work, handoff buttons let you switch to the next agent
- Skills, hooks, and MCP servers are fully supported
- MCP config location: **.vscode/mcp.json**

Download VS Code: code.visualstudio.com/download

Set up Copilot: docs.github.com/.../install-copilot-extension

Official docs: code.visualstudio.com/docs/copilot/overview

Cursor

A standalone code editor with AI built directly into it. Based on VS Code, so it will feel familiar if you have used VS Code before.

- Type slash commands like `/planner` directly in the chat to start a workflow
- Agents are treated as subagents, delegated automatically or invoked by name
- Skills load on demand, hooks run in the background
- MCP config location: `.cursor/mcp.json`

Download Cursor: cursor.com/download

Official docs: docs.cursor.com

Claude Code

Anthropic's AI coding assistant that runs in your terminal. Unlike VS Code and Cursor, it does not have a visual editor. You interact with it through text commands.

- Type slash commands like `/planner` in the terminal to start a workflow
- Agents are treated as subagents, delegated automatically or invoked by name
- Skills, hooks, and MCP servers all work out of the box
- MCP config location: `.mcp.json` at the project root

Setup guide: docs.anthropic.com/.../claude-code/setup

Part 2: Set Up the Apex Framework

Once your AI coding tool is installed, setting up the Apex Framework takes just a few minutes.

Step 1: Download the Apex Framework. Your lead or team will share a zip file containing the framework. Download it and extract the contents to a location on your computer.

Step 2: Copy the framework files into your project. From the extracted folder, copy these items into your project root folder: the `.claude` folder, the `.cursor` folder, the `.vscode` folder, and the `CLAUDE.md` file. These folders contain the agents, commands, skills, and configuration that power the framework.

Step 3: Open your project in your AI coding tool. Launch Cursor, VS Code, or Claude Code and open the project folder where you just copied the framework files.

Step 4 (Optional but recommended): Run the `/initialize-setup` command. This command scans your project, detects your tech stack and coding conventions, and writes your project-specific code standards into the `CLAUDE.md` file. While this step is not mandatory, it helps the framework understand your project better and provide more accurate assistance from the start.

That is it. The framework is now ready to use. Every developer on the team follows these same steps, and from this point on, everyone gets the same AI-powered context and behavior.

Part 3: MCP Servers (Optional Integrations)

MCP (Model Context Protocol) servers extend your AI coding tool's capabilities beyond code. They connect your tool to external services like Figma, Azure DevOps, Jira, and Chrome. These are optional but recommended if your workflow involves design files, user stories, or live site validation.

Figma Desktop MCP

Connects your AI tool directly to Figma design files. Extracts design tokens, component variants, layout structure, responsive breakpoints, and downloadable assets without leaving the IDE.

- Requires the Figma desktop app installed and updated
- Requires a Dev or Full seat on a paid Figma plan (Professional, Organization, or Enterprise)
- Open a Figma Design file, switch to Dev Mode, and enable the MCP server from the inspect panel
- No personal access token needed

```
{
  "mcpServers": {
    "figma-desktop": {
      "url": "http://127.0.0.1:3845/mcp"
    }
  }
}
```

Official docs: developers.figma.com/.../local-server-installation

Azure DevOps MCP

Fetches user stories, acceptance criteria, and task details directly from Azure DevOps. Reference a work item ID (e.g., #12345) in your prompt and the framework pulls the full context.

- Package: @azure-devops/mcp
- On first use, it opens a browser for Microsoft sign-in. No personal access token required.

```
{
  "inputs": [
    {
      "id": "ado_org",
      "type": "promptString",
      "description": "Azure DevOps organization name"
    }
  ],
  "servers": {
    "ado": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@azure-devops/mcp", "${input:ado_org}"]
    }
  }
}
```

Official docs: github.com/microsoft/azure-devops-mcp

Jira (Atlassian Rovo MCP Server)

Fetches Jira issues, acceptance criteria, and sprint context. Reference a Jira ticket (e.g., PROJECT-101) in your prompt and the framework pulls the full context.

- Uses browser-based OAuth 2.1 for authentication
- API token auth is available if an org admin enables it

```
{
  "servers": {
    "atlassian-mcp-server": {
      "url": "https://mcp.atlassian.com/v1/mcp",
      "type": "http"
    }
  }
}
```

Official docs: support.atlassian.com/rovo/getting-started

Chrome DevTools MCP

Gives your AI tool direct access to a running Chrome browser for DOM inspection, console logs, network requests, screenshot capture, and live site validation.

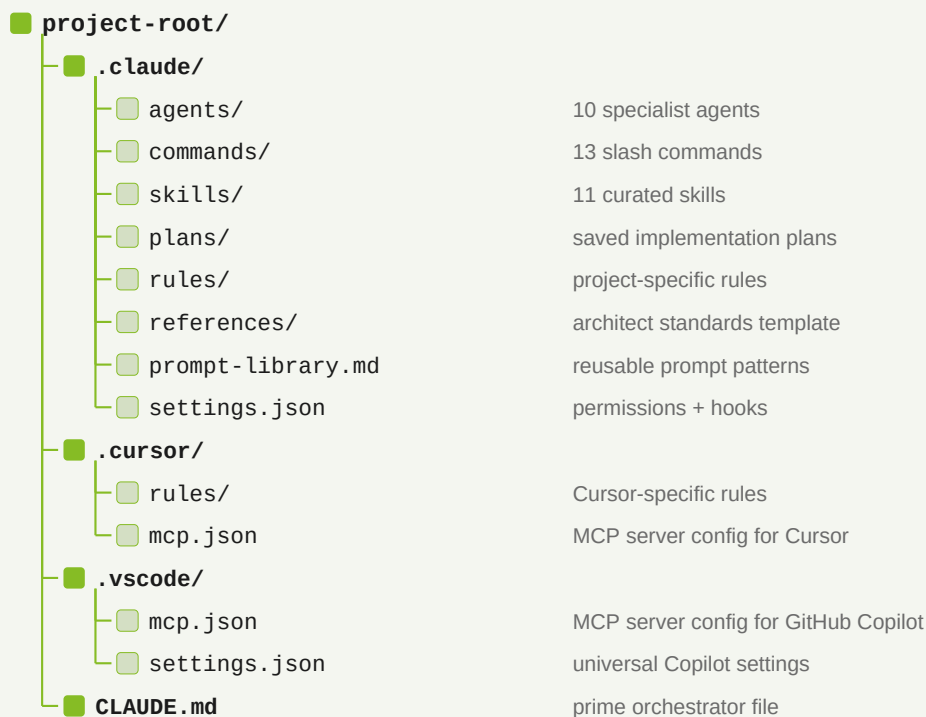
- Package: chrome-devtools-mcp
- No accounts or tokens required

```
{
  "mcpServers": {
    "chrome-devtools": {
      "command": "npx",
      "args": ["-y", "chrome-devtools-mcp@latest"]
    }
  }
}
```

Official docs: github.com/ChromeDevTools/chrome-devtools-mcp

MCP config file location varies by tool: **.cursor/mcp.json** for Cursor, **.vscode/mcp.json** for GitHub Copilot, **.mcp.json** (project root) for Claude Code.

FRAMEWORK STRUCTURE



10 SPECIALIST AGENTS

Agent	Role
Planner	Universal entry point. Classifies tasks, scans for reuse, produces implementation contracts
Backend Dev	Implements backend code following the approved plan strictly
Frontend Dev	Implements frontend code, templates, styling, and interactions
Code Reviewer	Reviews code for standards, security, performance. Reports only, never fixes
Test Specialist	Generates unit tests, targets 80% coverage, adapts to project test framework
Validation Tester	Validates deployed output: visual, functional, accessibility, design fidelity
Solutions Architect	Single decisive architectural recommendation with trade-off analysis
Code Explorer	Scans codebase for reusable components. Read-only, never modifies
Research Intel	Fetches platform documentation and technical guidance on demand
Docs Scribe	Creates architecture docs, ADRs, runbooks, and READMEs

DEVELOPMENT LIFECYCLE



Small and medium tasks run end-to-end automatically. Large tasks pause before deploy for confirmation.

13 SLASH COMMANDS

Command	Purpose
/planner	Plan any task before implementation. Always start here
/build	Trigger implementation from an approved plan
/review-code	Invoke code review on specific files or changes
/generate-junits	Generate unit tests for backend code
/generate-test-cases	Produce a structured test case table for a story or feature
/validate-implementation	Validate a deployed page or API against acceptance criteria
/debug-code	Systematic debugging with root cause analysis
/deploy-local	Build and deploy to local instance
/author-content	Author content via REST APIs. Fetches and uploads assets from design input
/publish-content	Replicate content to publisher. Requires explicit user confirmation
/initialize-setup	One-time project onboarding. Scans project, populates code standards
/generate-docs	Create architecture docs, ADRs, or runbooks
/commit-and-raise-pr	Commit changes and create a pull request

Slash commands are supported in Cursor and Claude Code. In GitHub Copilot, attach command .md files via # to activate workflows.

11 CURATED SKILLS (PROGRESSIVE DISCLOSURE)

Skill	Purpose
Utility Skills (framework core, stack-agnostic)	
framework-guidance	Design principles, anti-patterns, modification workflows for framework maintenance
skill-creator	Guide for creating new skills
skill-review	Audits skill documentation for accuracy and consistency
initialize-setup	Scans project to detect tech stack and conventions, populates CLAUDE.md
CMS-Specific Skills (applicable to any CMS)	
figma-context	Extracts design tokens, layout, variants, responsive breakpoints from Figma
storybook-patterns	Extracts patterns from Storybook stories, generates new stories
live-site-context	Extracts DOM structure and styles from live site URLs
story-context	Fetches requirements from Azure DevOps or Jira work items
planning-standards	Task classification, complexity, reusability analysis, implementation contracts
frontend-code-standards	Accessibility, security, testing, Storybook, component patterns
AEM-Specific Skills	
aem-code-standards	AEM platform reference: components, models, services, dialogs, templates, configs

Skills are loaded on demand, only when an agent needs them. Once the task completes, the context is discarded. The right knowledge, at the right time, for the right agent.